```
  end

  def create
    @comment = Comment.new(params[:comment])
    if @comment.save
      flash[:notice] = "Thanks for your comment!"
      if params[:remember_name]
        # Remember the commenter's name.
        cookies[:commenter_name] = @comment.author
      else
        # Delete cookie for the commenter's name cookie, if any.
        cookies.delete(:commenter_name)
      end
      redirect_to @comment.article
    else
      render action: "new"
    end
  end
end
```

Note that while for session values you set the key to `nil`, to delete a cookie value you should use `cookies.delete(:key)`.

Rails also provides a signed cookie jar and an encrypted cookie jar for storing sensitive data. The signed cookie jar appends a cryptographic signature on the cookie values to protect their integrity. The encrypted cookie jar encrypts the values in addition to signing them, so that they cannot be read by the end user. Refer to the API documentation for more details.

These special cookie jars use a serializer to serialize the assigned values into strings and deserializes them into Ruby objects on read.

You can specify what serializer to use:

```
Rails.application.config.action_dispatch.cookies_serializer = :json
```

The default serializer for new applications is `:json`. For compatibility with old applications with existing cookies, `:marshal` is used when `serializer` option is not specified.

You may also set this option to `:hybrid`, in which case Rails would transparently deserialize existing (`Marshal`-serialized) cookies on read and re-write them in the `JSON` format. This is useful for migrating existing applications to the `:json` serializer.

It is also possible to pass a custom serializer that responds to `load` and `dump`:

```
Rails.application.config.action_dispatch.cookies_serializer =
MyCustomSerializer
```

When using the `:json` or `:hybrid` serializer, you should beware that not all Ruby objects can be serialized as JSON. For example, `Date` and `Time` objects will be serialized as strings, and `Hash`es will have their keys stringified.

```
class CookiesController < ApplicationController
  def set_cookie
    cookies.encrypted[:expiration_date] = Date.tomorrow # => Thu, 20
Mar 2014
    redirect_to action: 'read_cookie'
  end

  def read_cookie
    cookies.encrypted[:expiration_date] # => "2014-03-20"
  end
end
```

It's advisable that you only store simple data (strings and numbers) in cookies. If you have to store complex objects, you would need to handle the conversion manually when reading the values on subsequent requests.